

AMIRIS Germany2027 - Q&A Deep Dive

Real evidence checked against AMIRIS's own files, actual schema definitions, and researched external sources - answering every follow-up question from the build review

Muideen Oladayo Ajiboye | 12 August 2026

1. Historical Fuel and Carbon Prices, 2015 to Today (Real Data)

Checked directly, not recalled from memory. Anchor points below are real, sourced figures - some years only have a single reference point rather than a dense monthly series, since that is what open sources actually publish.

Year	Hard coal (USD/t)	Brent oil (USD/bbl)	Gas TTF (EUR/MWh)	EUA carbon (EUR/t)
2015	~57.5	52	~20 (pre-crisis range)	5.20-7.60 (2014-17 range)
2018	in the 50-100 band (exact year not published)	71	~20	~15
2020	in the 50-100 band	42 (COVID crash)	~5 (mid-2020 low)	~25
2022	384.17 (year avg; intraday peak ~457.80 in Sep)	100	~345 (all-time peak, Mar 2022)	105.73 peak (Feb 2023)
2024	143.55	80-82	falling, off >90% from peak	in the 80s
2026 (11 Aug)	129.30	~85 (mid-2026 spot)	60.00	82.40

Lignite has no equivalent row - confirmed, not assumed:

Lignite is not internationally traded. Clean Energy Wire's Germany coal factsheet states it directly: "Because lignite is humid and heavy, it is too expensive to transport over long distances, and so normally it is burned in power stations near the mines." There is no benchmark analogous to API2/Brent/TTF for it. The best available proxy is Oeko-Institut's (2017) estimated marginal fuel cost of roughly 1.50-3.60 EUR/MWh (plus about 2 EUR/MWh variable O&M) for German lignite plants - this is why AMIRIS's own scenario (and this build) treats lignite fuel cost as a flat internal estimate (5.00 EUR/MWh) rather than a market-quoted series.

Big-picture pattern worth keeping in mind when comparing your Germany2027 build's converted prices against real history: 2022 was a genuine outlier across every fuel simultaneously (post-COVID demand rebound colliding with Russia's gas-supply cuts), not a representative 'normal' year. Prices by 2024-2026 settled well below the 2022 peak but generally above pre-2021 levels, especially for carbon.

Sources for Section 1

- World Bank Commodity Markets Pink Sheet: <https://thedocs.worldbank.org/en/doc/18675f1d1639c7a34d463f59263ba0a2-0050012025/world-bank-commodities-price-data-the-pink-sheet>
- FRED - Global Price of Coal, Australia: <https://fred.stlouisfed.org/series/PC0ALAUUSDA>
- FRED - Crude Oil Prices: Brent: <https://fred.stlouisfed.org/series/DCOILBRETEU>
- EIA - Europe Brent Spot Price FOB: <https://www.eia.gov/dnav/pet/hist/rbrtem.htm>
- Trading Economics - Coal: <https://tradingeconomics.com/commodity/coal>
- Trading Economics - EU Natural Gas (TTF): <https://tradingeconomics.com/commodity/eu-natural-gas>
- Trading Economics - EU Carbon Permits (EUA): <https://tradingeconomics.com/commodity/carbon>
- Clean Energy Wire - Coal in Germany factsheet: <https://www.cleanenergywire.org/factsheets/coal-germany>
- ScienceDirect - How marginal is lignite?: <https://www.sciencedirect.com/science/article/pii/S0301421520302305>
- DEHSt/EEEX - EU ETS Auctioning: https://www.dehst.de/EN/Topics/EU-ETS-1/EU-ETS-1-Information/Auctioning/auctioning_node.html

2. Were AMIRIS's Own Outage Figures Constant 2015-2019?

Q: Were they constant from 2015 to 2019?

No. Checked directly by averaging each year's actual file:

File	2015	2016	2017	2018	2019
Lignite outage (mean)	0.196	0.244	0.247	0.245	0.352
Hard coal outage (mean)	0.452	0.453	0.457	0.443	0.612
Natural gas outage	flat 0.03 (a number, not a file)	flat 0.03	flat 0.03	flat 0.03	a real file, mean 0.402

Two different things are changing here, not one:

First, the outage LEVEL genuinely differs year to year (real historical plant-maintenance patterns - 2019 shows visibly higher outage rates for both lignite and hard coal than 2015-2018). Second, and more strikingly, natural gas's outage handling changed STRUCTURE in 2019: for 2015-2018, AMIRIS's own scenarios use a single flat number (3%) typed directly into Conventional.yaml, with no timeseries file at all. Only in 2019 did AMIRIS's own example switch natural gas to a full weekly-resolution outage file. This build used the 2019 approach (a real file) since it is AMIRIS's own most detailed and most recent convention, and it is the one this whole project's earlier work is built around.

Practical consequence: this build's Germany2027 scenario reuses the 2019 file, redated. This is one real historical weather/maintenance year's pattern, not a multi-year average or a 'typical' year - worth stating explicitly as a limitation. A stronger future version could average outage patterns across all five available years (2015-2019) to smooth out any one year's idiosyncrasies, at the cost of losing the real week-to-week structure any single year has.

3. Wind Offshore and Run-of-River Profiles: Constant 2015-2019?

Q: Were they constant, and if not, how did you come up with the value used for 2027?

No - all four renewable profiles checked show genuine year-to-year weather variation:

Profile	2015	2016	2017	2018	2019
Wind offshore (mean CF)	0.283	0.332	0.394	0.431	0.371
Run-of-river (mean CF)	0.292	0.413	0.386	0.329	0.345
Biomass (mean CF)	0.587	0.637	0.649	0.636	0.591
Other renewables (mean CF)	0.187	0.274	0.347	0.302	0.368

How the 2019 value was chosen: not statistically derived, and not claimed to be 'typical' - it is simply AMIRIS's own most recent and most structurally detailed example year (already established earlier in this project as the richest of the five: separate must-run/outage files, 18 individual storage units, versus the simpler structure of 2015-2018). Reusing it for 2027 means this build's wind/hydro/biomass weather assumption is literally '2019's real weather,' relabelled. The offshore wind figure happened to get an independent sanity check (Section 1 finding: its 37.1% implied annual capacity factor sits inside the real German fleet's 37-45% range) but that check was fortunate, not the reason 2019 was chosen - the reason was simply 'most detailed available example.'

Honest limitation:

A single historical year, however recent, is not a forecast of 2027's actual weather - it is a placeholder weather assumption, exactly like AMIRIS's own example scenarios already are for their own target years. If a more defensible 2027 weather assumption is wanted later, averaging profiles across all five years, or sourcing a genuine weather-year projection (e.g. via renewables.ninja, as already discussed for the offshore cross-check), would both be stronger choices than a single arbitrary year.

4. other_res_profile.csv, biomass_profile.csv, and Brainpool's Missing Biomass Row

Q: I don't understand the explanation for other_res_profile.csv and biomass_profile.csv, and it seems Brainpool has no biomass capacity - how was that handled?

Here is the fuller picture, worked from AMIRIS's own real Germany2019 file directly rather than described abstractly.

First, on Brainpool: you're right that there is no row literally labelled 'Biomass' in the Capacity sheet. Brainpool instead uses the standard German statistical convention, where biomass, biogas, landfill/sewage gas, and geothermal are all lumped into one catch-all row: 'Other Renewables' ('Sonstige EE'), 8.25 GW capacity / 49.85 TWh generation. That row IS the biomass capacity, under a broader label - Brainpool simply never breaks it down further.

Second, on AMIRIS's own side: its Germany2019 example actually keeps TWO separate categories that Brainpool's one row bundles together. A Biogas agent (Id 52) with 7,833 MW, using biomass_profile.csv (a dispatch shape, not a weather capacity factor - biomass plants are dispatched more like a controllable fuel-burning plant than a weather-dependent one). And a second, much smaller VariableRenewableOperator (Id 53, EnergyCarrier: 'Other') with only 454 MW, using other_res_profile.csv (a genuine weather-driven capacity factor, for whatever residual technology - likely a small amount of geothermal or similar - AMIRIS's own example keeps separate from biomass).

	AMIRIS's own Germany2019 split	Share of the combined total
Biogas (Id 52)	7,833 MW	94.5%
'Other' (Id 53)	454 MW	5.5%
Combined	8,287 MW	100%

Why the whole 8.25 GW was mapped onto Biogas alone:

AMIRIS's own real numbers above show biomass is genuinely the dominant component (94.5%) of this combined category - so treating Brainpool's entire 'Other Renewables' figure as biomass is a small, quantified simplification (missing only the ~5.5% 'Other' sliver), not a rough guess. The other_res_profile.csv file was still built from AMIRIS's own data during the build (see the earlier documentation) but is not currently wired to any agent in this version - it exists on disk as a documented option if a future version wants to split out that small remainder explicitly.

5. Load Demand: The Calculation, Explained Again From Scratch

Q: Explain again how you calculated the demand and came up with the Excel numbers.

Step by step, with the real numbers this build actually used:

Step 1 - read four numbers straight from Brainpool's Capacity sheet, Section C (Gross Electricity Demand): Inflexible base demand 604.85 TWh, Electrolysis 14.97 TWh, Electric mobility 17.81 TWh, Heat pumps 18.69 TWh. These are Excel numbers, not calculated by this build - they came directly out of the spreadsheet cells.

Step 2 - add the four together: $604.85 + 14.97 + 17.81 + 18.69 = 656.32$ TWh. This is the single annual total the whole year's hourly load curve has to sum to.

Step 3 - hand that total to demandlib, a real Python implementation of BDEW's Standard Load Profile (Standardlastprofil) methodology. The call made was:

```
slp = bdew.ElecSlp(2027)
profile = slp.get_scaled_power_profiles({'h0': 656_320_000_000}) # value in Wh
```

Internally, demandlib does the following, none of it invented for this build - this is the real, published BDEW methodology: it holds a normalised household ('H0') load shape, built from twelve representative daily curves (one for each combination of weekday/Saturday/Sunday and four seasons), with an additional smooth day-of-year adjustment specific to H0 so the shape drifts continuously through the year rather than jumping between four blocks. It stamps that shape across every day of 2027 at 15-minute resolution (35,040 points for the year), then multiplies every single point by one scaling factor equal to the target annual total divided by the shape's own reference total - so the SHAPE (relative ups and downs) comes from BDEW's real household data, while the SIZE comes from Brainpool's 656.32 TWh.

Step 4 - resample from 15-minute to hourly (AMIRIS's expected resolution) by averaging each hour's four 15-minute values, then convert from kW to MW.

Verification actually performed, not just claimed:

After building the file, its 8,760 hourly values were summed back up and compared against the target: the result was 656.3167 TWh against a target of 656.3167 TWh - an exact round-trip match, confirming the scaling step worked correctly.

The known weakness of this V1 approach (already flagged in the build documentation, and the reason a V2 is planned): H0 is a HOUSEHOLD shape. Applying it to the entire 656.32 TWh - which includes industrial demand, EV charging, and heat pumps, none of which actually follow a household's evening ramp - produces a load curve peakier than reality. This build's peak came out at 138.1 GW, noticeably above Brainpool's own stated Annual Peak Load of 119.0 GW, and diagnostic work traced most of this scenario's remaining supply-shortage hours directly to that evening peak.

6. Block Size, Cycling Cost, and Efficiency Ranges

Q: How does block size work, and what informed AMIRIS's choices (e.g. 500 for lignite)?

AMIRIS's own schema.yaml defines BlockSizeInMW plainly: "Defines the typical power capacity of a PowerPlant within the Portfolio to generate." Mechanically: PredefinedPlantBuilder takes the fleet's total installed capacity and divides it by the block size to work out how many individual simulated plants to create. Lignite's 13,927 MW at a 500 MW block size becomes roughly 28 separate simulated plants; Oil's 9,790 MW at a 100 MW block size becomes roughly 98. Each of those simulated plants then gets its own efficiency value (see below), so the fleet bids into the market as many small steps in the merit order rather than one giant undifferentiated block.

AMIRIS's documentation does not state an explicit rationale for its specific chosen numbers (500 for lignite, 300 for hard coal, 200 for gas, 100 for oil), so here is a reasoned, not officially confirmed, interpretation: these numbers track roughly the scale of a single real generating unit for that technology in the German fleet. Large modern lignite units (e.g. the BoA units at Niederaussem) run close to 1,000 MW, hard coal supercritical units commonly 300-800 MW, and Germany's gas fleet is more heterogeneous with many smaller CHP-linked units pulling the representative size down toward 200 MW; oil-fired capacity today is almost entirely small peaking/reserve units, consistent with 100 MW.

Q: What is cycling cost?

AMIRIS's own definition: "Additional cycling costs for conventional power plants in Euro per Megawatt, i.e. costs due to plant start up." It represents the real extra cost a thermal plant incurs every time it is shut down and later restarted (thermal stress, fuel used to reheat boilers, maintenance wear) - distinct from OpexVarInEURperMWh, which is the cost of running per MWh produced while already online. Every fuel type in this build uses AMIRIS's own value of 0.0 for this field (matching AMIRIS's own Germany2019 example throughout), meaning cycling costs are not currently represented in either scenario - a simplification inherited from AMIRIS's own examples, not something introduced by this build.

Q: The efficiency figures are ranges (Minimal/Maximal) - how are they applied across the hours?

They are not applied across hours at all - that's a natural but incorrect reading of 'range.' The Minimal-to-Maximal range is applied ACROSS THE INDIVIDUAL SIMULATED PLANTS (the blocks described above), not across time. When PredefinedPlantBuilder splits a fuel type's total capacity into individual blocks, each block is assigned a different, fixed efficiency value spread across the given range - so hard coal's 0.339-0.492 range means the least efficient simulated hard coal block runs at roughly 33.9% and the most efficient at roughly 49.2%, creating real internal competition within one fuel type (the most efficient blocks have a lower marginal cost and clear the market first). Once assigned, a given block's efficiency stays fixed for the whole simulated year - it does not change hour to hour. (Technical note: AMIRIS's schema does allow Efficiency to be supplied as a time series rather than a flat number, which would support a genuinely seasonal efficiency effect - e.g. thermal plants running slightly less efficiently in summer heat - but this build, like AMIRIS's own examples, uses flat constant values, so that time-varying option is not in use here.)

7. Value of Lost Load

Q: What is Value of Lost Load (I saw it in the Demand.yaml file)?

AMIRIS's own schema.yaml description: "The energy price on a market with scarcity equals the maximum allowed price (scarcity price)." In plain terms: it is the price the simulation charges whenever demand cannot be fully met by available supply - a deliberately high ceiling price (3,000 EUR/MWh in this build, matching AMIRIS's own convention) representing how much value is lost when the lights genuinely go out for lack of generation. It functions as the market's

price cap: whenever you saw prices sitting exactly at 3,000 EUR/MWh in the results (the shortage hours discussed at length in the build documentation), that is this exact mechanism firing - the market literally could not find enough supply to meet the DemandTrader's request that hour, so the price clamped to this ceiling instead of clearing normally.

8. RenewablesAndPolicy.yaml, Explained Properly

This file does two jobs at once: it defines every renewable/biomass GENERATOR agent (how much capacity, what hourly shape it follows), and it defines the SUBSIDY SYSTEM those generators are paid through. Four kinds of agent appear in it:

Agent type	Role	In this build
VariableRenewableOperator	A weather-driven generator: bids its available output (capacity x hourly capacity-factor) into the market, usually near-zero cost.	Solar Openfield, Solar Rooftop, Wind Onshore, Wind Offshore, Run-of-River - five of the six agents.
Biogas	A dispatchable generator following a pre-set hourly DispatchTimeSeries rather than a weather shape.	The Biomass agent (Id 52) - the sixth.
RenewableTrader / SystemOperatorTrader	The market-facing intermediary each generator sells through. RenewableTrader (Id 11) handles MPVAR-marketed generators; SystemOperatorTrader (Id 13) handles FIT-marketed ones.	One of each, shared across all six generators depending on their subsidy type.
SupportPolicy	The subsidy administrator (Id 90). Holds every PolicySet's rate and pays out the FIT/MPVAR top-up described in Section 9 below.	One agent, carrying the six rates from the build documentation's Part 3 table.

Each VariableRenewableOperator/Biogas agent's Attributes carry: PolicySet (which subsidy rate applies to it), SupportInstrument (FIT or MPVAR), EnergyCarrier (its technology label), InstalledPowerInMW (capacity), OpexVarInEURperMWh (its bidding floor, 0.0 throughout), and YieldProfile or DispatchTimeSeries (which timeseries file drives its hourly output).

9. MPVAR, FIT, and LCOE Explained, With Real Sources

Q: What is MPVAR and FIT?

FIT (Feed-in Tariff): the plant receives a fixed, guaranteed EUR/MWh for every unit it feeds into the grid, completely independent of the market price that hour. The legal basis is EEG Section 49, which sets both the rate and a degression schedule (currently a roughly 1% reduction every six months) that gradually lowers the rate for newly-built plants over time.

MPVAR (variable Market Premium): the plant sells directly into the wholesale market at whatever price that hour clears at, then separately receives a top-up payment calculated against a reference cost figure (the LCOE, below). This is the mechanism EEG requires for larger installations (ground-mounted PV over 1 MW, most wind) via competitive auctions run by the Bundesnetzagentur (BNetzA), rather than a flat statutory rate.

Q: What is LCOE?

Levelised Cost of Electricity - a single EUR/MWh figure meant to represent a generator's full cost of producing electricity averaged over its entire lifetime (construction, financing, operation, maintenance, eventual decommissioning, all divided by total lifetime energy output). In AMIRIS's MPVAR mechanism specifically, the LCOE figure is the reference cost the top-up payment is calculated against: if the market price a plant actually receives falls short of its LCOE, the premium tops it up toward that reference level. In this build, the MPVAR agents' 'Lcoe' figures were set from real BNetzA auction-cleared prices, which function as a market-discovered proxy for LCOE (an auction bid is, roughly, a bidder's own LCOE estimate plus a margin).

Sources for Section 9

- EEG 2023 official law text (gesetze-im-internet.de): https://www.gesetze-im-internet.de/eeg_2014/BJNR106610014.html
- BNetzA - Ausschreibungen (tenders) overview: <https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/Ausschreibungen/start.html>
- BNetzA - Wind Onshore results, 1 Feb 2026 round: https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/Ausschreibungen/Wind_Onshore/BeendeteAusschreibungen/2026/GT01022026/start.html
- BNetzA - Solar (ground-mounted) tender, 1 Mar 2026: <https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/Ausschreibungen/Solaranlagen1/01032026/artikel.html>
- BNetzA - Solar rooftop tender, Feb 2026: <https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/Ausschreibungen/Solaranlagen2/Feb2026/start.html>
- BNetzA - 2026 Hoechstwerte press release: https://www.bundesnetzagentur.de/SharedDocs/Pressemitteilungen/DE/2025/20251216_Hoechstwerte.html
- MaStR - public bulk data download: <https://www.marktstammdatenregister.de/MaStR/Datendownload>
- renewables.ninja - free API registration: <https://www.renewables.ninja/register>

10. Markup Band

Q: What is the markup band, and how is it determined?

AMIRIS's own schema.yaml is explicit and worth quoting directly: minMarkup is "the lower bound of the interpolation for changes to the marginal costs," and maxMarkup is "the upper bound," both in EUR/MWh, both defaulting to 0. The actual bid a ConventionalTrader submits is its plant's marginal cost (fuel/efficiency + carbon x emissions factor + opex) PLUS a markup drawn from somewhere inside this band - so the band defines how far a plant is allowed to bid above or below its own true cost.

Why the bands differ by fuel type, reading the actual numbers used: nuclear (not used in this 2027 build, but present in AMIRIS's own examples) allows a deeply negative band (-150 to -100), reflecting that shutting an inflexible plant down and restarting it is far more expensive than paying to keep it running at a loss for a few hours - so it 'pays' to stay on the grid even at a loss. Lignite's -60 to 0 band reflects a similar but less extreme inflexibility. Hard coal's -15 to 5 and gas's -10 to 10 bands are close to symmetric around zero, reflecting more flexible plants with less structural incentive to bid negative. Oil's 0 to 0 (no markup at all, bids exactly at marginal cost) reflects its role as expensive, rarely-dispatched peaking capacity, where there is no real strategic reason to bid away from true cost.

AMIRIS's schema does not document the exact formula for WHERE inside the band a given hour's markup lands (whether it's a function of forecast scarcity, a random draw, or something else) - that logic lives inside the Java simulation engine itself, not in the schema's own metadata, so this build carried over AMIRIS's own example bands unchanged rather than inventing new ones, consistent with every other AMIRIS-own gap-fill in this project.

11. Must-Run vs. Outage, Worked Through With Real Natural Gas Numbers

Both factors reduce how much of a fleet's capacity is genuinely available to bid, but in opposite directions. Outage = the share that CANNOT run (broken down, under maintenance) - a ceiling on what's available. Must-run = the share that CANNOT be turned off (forced to run and sell regardless of price) - a floor on what's actually offered. AMIRIS's own schema.yaml for MustRunFactor: "Share of the installed capacity that must run and may not shut down."

Real natural gas numbers from AMIRIS's own Germany2019 file, first three weeks of January:

Week	Outage factor	Must-run factor	Reading
Jan 1-7	0.03 (flat, carried from the prior year)	0.081	3% of the gas fleet is broken/under maintenance and cannot produce; separately, 8.1% is forced online regardless of price.
Jan 8-14	0.03	0.101	Outage unchanged; must-run rose to 10.1% - e.g. more CHP plants forced on for winter heat demand.
Jan 15-21	0.03	0.144	Must-run keeps climbing (14.4%) - outage still flat.
Jan 22-28	0.03	0.123	Must-run eases slightly (12.3%).

The two numbers are independent and can both apply to the same hour simultaneously: in the Jan 15-21 week, 3% of gas capacity is simply gone (outage) while a separate 14.4% is compelled to run no matter what price it clears at (must-run) - the remaining ~82.6% is the fleet's genuinely price-responsive, biddable capacity that week.

A subtlety worth flagging directly:

For 2015-2018, AMIRIS's own examples used a single flat outage number (0.03) for natural gas with no file at all - only 2019 switched to a real weekly file (which is what this build reused, redated to 2027). December 2019's outage file also shows a real, sharp escalation toward year-end - 17.6% by Dec 10, climbing to 51.5% by Dec 31 - a genuine feature of AMIRIS's own 2019 data, not an artifact introduced by this build.

12. scenario.yaml: RandomSeed and OEONearestConcept

Q: What does RandomSeed mean?

A standard simulation concept, not AMIRIS-specific: computers cannot generate truly random numbers, only pseudo-random sequences that are entirely determined by a starting number called a seed. AMIRIS uses randomness internally (for example, in exactly where within a markup band a bid lands, per Section 10). Setting RandomSeed: 1 means every run of this exact scenario, on any machine, produces the identical 'random' sequence and therefore identical results - essential for a thesis, since it means your results are reproducible and not silently different each time you rerun the simulation.

Q: What does OEONearestConcept mean?

This one is genuinely worth knowing where it comes from: OEO stands for the Open Energy Ontology (openenergyplatform.org), a real, published, formal vocabulary for describing energy-system concepts in a machine-readable way, maintained by an academic/open-data consortium. Every field in AMIRIS's schema.yaml that carries an 'isAbout' or 'OEONearestConcept' tag (e.g. OEO_00020076 for 'markup', confirmed directly against the OEO ontology) is linking that specific AMIRIS parameter to the closest matching standardised concept in this external ontology. It has no effect on the simulation itself - it exists purely so that AMIRIS's data can, in principle, be automatically compared or combined with other energy models and datasets that also tag their fields against the same ontology.

Sources for Section 12

- Open Energy Ontology (OEO) home: <https://openenergyplatform.org/ontology/>
- OEO ontology viewer: <https://openenergyplatform.org/viewer/oEO/>

13. Storage.yaml, Every Term Explained

AMIRIS's own schema description for the whole device block is worth starting with: "A generic device representing any kind of electrical flexibility, e.g., pumped-hydro storages with inflow, reservoir storages, heat pumps, electric vehicle fleets, or load-shifting portfolios." Every term below is AMIRIS's own definition, not a paraphrase.

Field	What it means (AMIRIS's own definition)
GrossChargingPowerInMW	"Gross maximum power for charging in MW drawn from the grid (external power)" - how fast it can draw power from the grid to charge.
NetDischargingPowerInMW	"Net maximum power from discharging provided to grid (external power)" - how fast it can deliver power back to the grid.
ChargingEfficiency	The share of grid power drawn during charging that actually ends up stored (the rest is lost as heat/conversion loss).
DischargingEfficiency	The share of stored energy that actually reaches the grid when discharging (the rest is lost the same way).
EnergyContentUpperLimitInMWH	"Maximum internal energy content of the flexibility device" - the size of the storage 'tank', in MWh.
InitialEnergyContentInMWH	The storage's starting energy level when the simulated year begins. This build set each device to roughly half-full at the start.
EnergyResolutionInMWH	"Resolution of the energy discretisation; lower values represent a higher resolution and better precision of planning, but come with (quadratically) higher calculation efforts." AMIRIS plans storage dispatch over a discretised grid of possible energy states, not a continuous value - this sets how fine that grid is.
PlanningHorizonInHours	"Foresight time length in hours. Must be smaller or equal to that of the contracted Forecaster." How far ahead the storage's internal planning looks when deciding how to use its limited energy.
SchedulingHorizonInHours	"Number time length each created schedule is viable in hours; should be significantly larger than the flexibility's energy-to-power ratio." How long a dispatch plan, once made, stays in force before being recalculated.
Assessment: Type	The storage's underlying objective. MIN_SYSTEM_COST: "Minimises total system costs using a merit order sensitivity forecast." MAX_PROFIT: "Maximises own profits using a merit order sensitivity forecast." This build used MIN_SYSTEM_COST for Pumpspeicher and Reservoir Hydro (matching AMIRIS's own convention for large legacy hydro, run as grid-serving assets) and MAX_PROFIT for the Battery (matching AMIRIS's own convention for merchant-style assets, run to make money).
Bidding: Type	How that objective becomes an actual bid. ENSURE_DISPATCH: "Ensures planned dispatch is fulfilled by bidding at technical price limits" - bids aggressively enough to guarantee the planned schedule executes. STORAGE_CONTENT_VALUE: "Uses estimated value changes of storage content to calculate bidding price" - bids based on what the stored energy is estimated to be worth right now versus later.
StateDiscretisation: Type (STATE_OF_CHARGE)	Declares that the discretised planning grid described above is organised by state of charge (how full the storage is), the standard approach used throughout AMIRIS's own examples.

One real limitation worth flagging for future refinement:

AMIRIS's schema also defines a *NetInflowPowerInMW* field ("Net inflow energy into the flexibility device... positive values are inflows") - built specifically for cases like a reservoir with natural water inflow from rain/snowmelt, independent of pumped charging. This build's Reservoir Hydro agent does not use it (defaults to 0, meaning the reservoir is only ever filled by deliberate pumping, never by natural inflow) - a simplification worth revisiting if reservoir hydro's role in the results turns out to matter.

14. The Contracts Folder, Every File Explained

Every contract file shares the same two-part shape: an AgentGroups block that names which agent IDs play which role (using YAML anchors, the '&name' syntax, so the same list can be referenced repeatedly below without retyping it), and a Contracts block listing individual message exchanges between those roles. Each contract entry has five parts: SenderId/ReceiverId (which agents exchange this message), ProductName (what kind of message - a bid, a price forecast, an award, a payment), FirstDeliveryTime (when the first exchange happens, in seconds relative to each simulated hour - negative numbers mean 'before' that hour, since forecasts and bid preparation happen ahead of the actual market clearing), and Every (how often it repeats - 1 hour for most day-ahead-market messages, 1 month for support payments, 1 year for annual registration-type messages).

File	What it wires together
conventionals.yaml	The full conventional-plant pipeline: PlantBuilder -> Operator -> Trader -> Exchange, plus each operator's forecast requests to the FuelsMarket and CarbonMarket for pricing, and the exchange's gate-closure/award/payout cycle back down.
demand.yaml	The DemandTrader's own gate-closure, forecast, bid, and award cycle with the exchange - structurally identical to a generator's cycle, just on the buying side.
renewables_renewableTrader.yaml	Every MPVAR-marketed renewable agent's registration, forecasting, and bidding cycle, all routed through the shared RenewableTrader (Id 11).
renewables_systemOperator.yaml	The same cycle for FIT-marketed agents, routed through the shared SystemOperatorTrader (Id 13) instead.
storage.yaml	Each storage agent's forecast registration with the Forecaster - shorter than the other files since GenericFlexibilityTrader agents bid directly to the exchange without an intermediate trader.
supportPolicy.yaml	The subsidy settlement cycle: renewable/biomass agents report their actual yield to the SupportPolicy agent, which calculates and pays out the FIT/MPVAR top-up monthly, plus an internal monthly MarketValueCalculation the policy agent runs on itself to determine the reference market price MPVAR top-ups are measured against.

Why this two-layer design (agent files define WHAT exists; contract files define WHO TALKS TO WHOM) matters practically: it is exactly why this build's simplification work was safe to do by trimming ID lists rather than rewriting contract logic from scratch (Part 5 of the build documentation) - the message-passing rules themselves (what a forecast request looks like, when a payment happens) are completely generic and identical regardless of which specific agents are plugged into which role.